



Politechnika Wrocławska

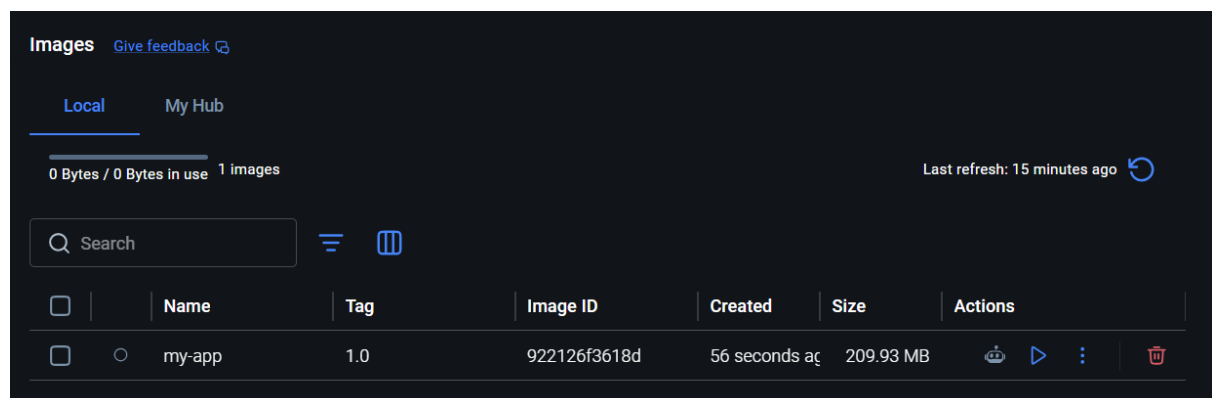
Praktyka programowania
Docker

Kacper Kruszelnicki 280150

1 Konteneryzacja aplikacji

```
PS C:\Users\krusz\GitHub\Programming-Practice\06-Docker> docker build -t my-app:1.0 .
[+] Building 5.8s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 484B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim 0.6s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.10-slim@sha256:fa184fce49c170a8b1032a4f752f9fe 0.0s
=> => resolve docker.io/library/python:3.10-slim@sha256:fa184fce49c170a8b1032a4f752f9fe 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 545B                                       0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt requirements.txt             0.0s
=> [4/5] RUN pip install -r requirements.txt                       3.9s
=> [5/5] COPY . .                                                  0.1s
=> exporting to image                                              1.0s
=> => exporting layers                                              0.7s
=> => exporting manifest sha256:982b1f20c33064ac074516d7d04f5a89e1b6530f6e24c9ef5981224 0.0s
=> => exporting config sha256:15ef8efe333c890bcc40d66821dd3cbdf283840e123c39d4d52d87147 0.0s
=> => exporting attestation manifest sha256:a453e249f99175e8e57d62ebd0fd988aa7e1b8b0b71 0.0s
=> => exporting manifest list sha256:922126f3618dab7ccb4ed8c521a0cd4a15e4600010ed7529f5 0.0s
=> => naming to docker.io/library/my-app:1.0                      0.0s
=> => unpacking to docker.io/library/my-app:1.0                    0.3s
```

Rysunek 1: Budowanie obrazu



Rysunek 2: Pokazanie zbudowanego obrazu w Docker Desktop

```
PS C:\Users\krusz\GitHub\Programming-Practice\06-Docker> docker run -p 5000:5000 my-app:1.0
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 939-283-083
```

Rysunek 3: Uruchomienie aplikacji w terminalu

ConnectionError

redis.exceptions.ConnectionError: Error -2 connecting to redis:6379. Name or service not known.

Traceback (most recent call last)

File "/usr/local/lib/python3.10/site-packages/redis/connection.py", line 1033, in connect_check_health

```
sock = self._connect()
```

File "/usr/local/lib/python3.10/site-packages/redis/connection.py", line 1545, in _connect

```
for res in socket.getaddrinfo(
```

File "/usr/local/lib/python3.10/socket.py", line 967, in getaddrinfo

```
for res in _socket.getaddrinfo(host, port, family, type, proto, flags):
```

During handling of the above exception, another exception occurred:

File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 1536, in __call__

```
return self.wsgi_app(environ, start_response)
```

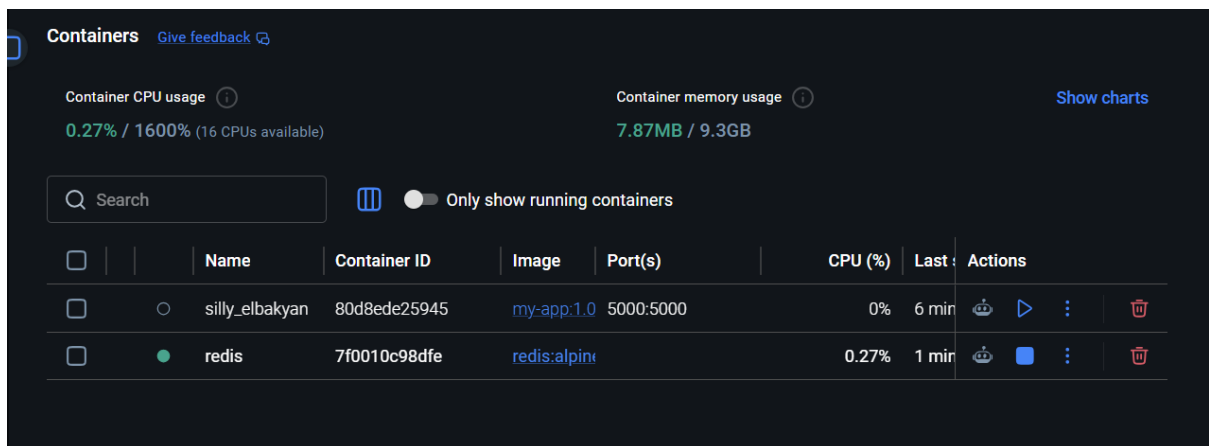
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 1514, in wsgi_app

```
response = self.handle_exception(e)
```

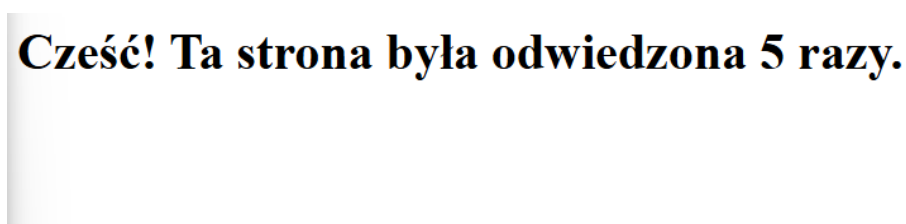
Rysunek 4: Po wejściu na adres pojawił się następujący błąd. Spowodowany przez brak połączenia z instancją bazy danych.

```
PS C:\Users\krusz\GitHub\Programming-Practice\06-Docker> docker run -d --name redis --network my-app-net redis:alpine
Unable to find image 'redis:alpine' locally
alpine: Pulling from library/redis
b1f21fb9c363: Pulling fs layer
b1f21fb9c363: Download complete
21ee05ca5806: Pull complete
6a0ac1617861: Pull complete
9b7c44b6f30a: Pull complete
259fe4442447: Pull complete
4f4fb700ef54: Pull complete
7c7d412e5672: Download complete
710e2faa951e: Download complete
Digest: sha256:09160599abd229764c0fb44cb6be640294e1d360a54b19985ab4843dcf2d90f1
Status: Downloaded newer image for redis:alpine
7f0010c98dfea1d17fcdf2e319c7d80f76bf8bb0c2329e486cc0df8d7296bfa9
```

Rysunek 5: Uruchomienie kontenera.

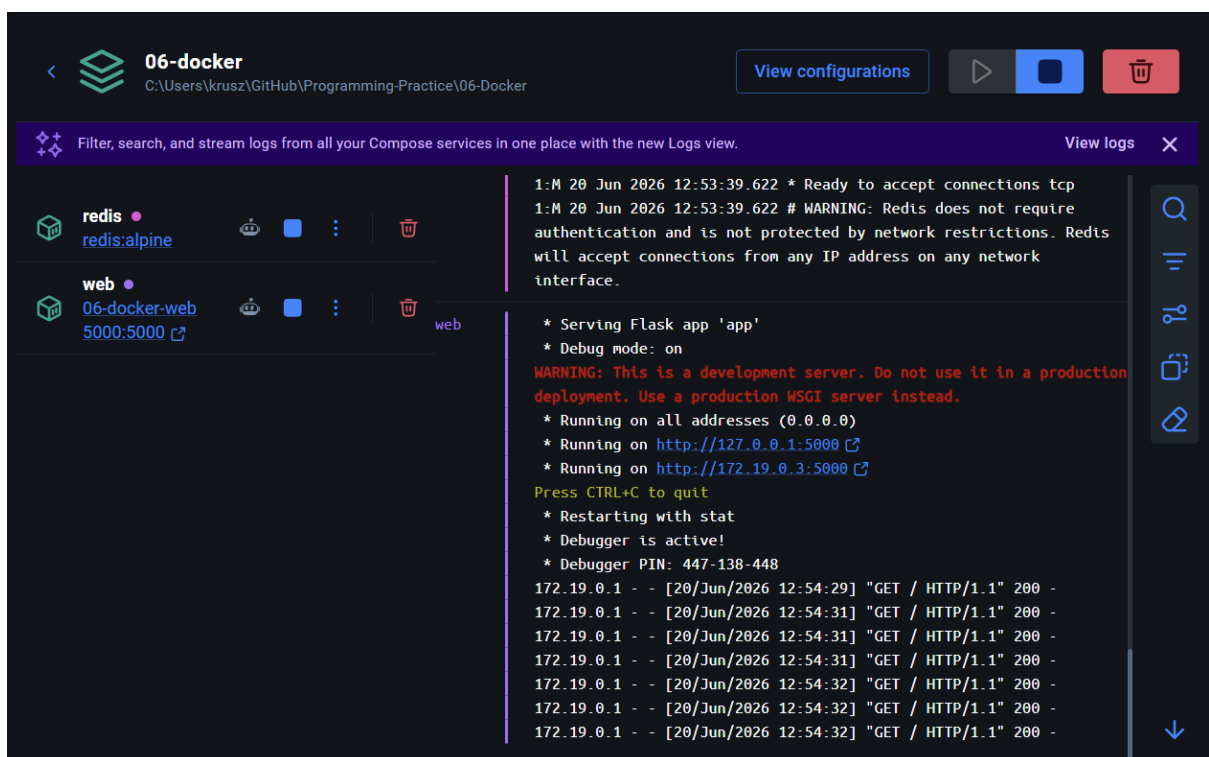


Rysunek 6: odpalony kontenerem w widoku z Docker Desktop.



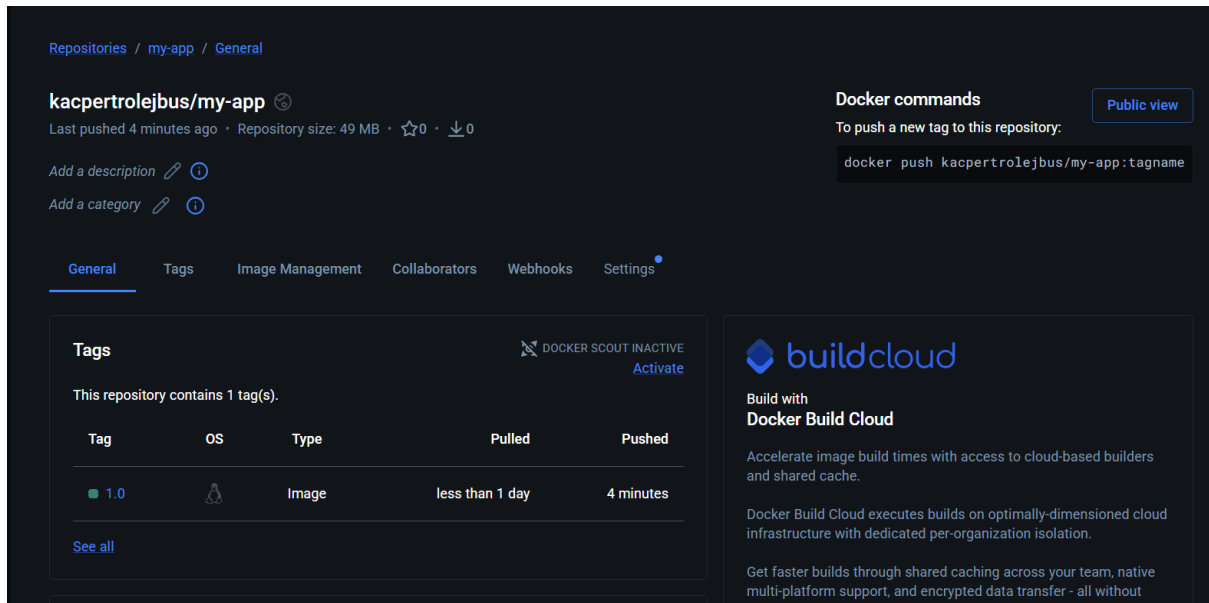
Rysunek 7: Działająca aplikacja.

2 Docker Compose

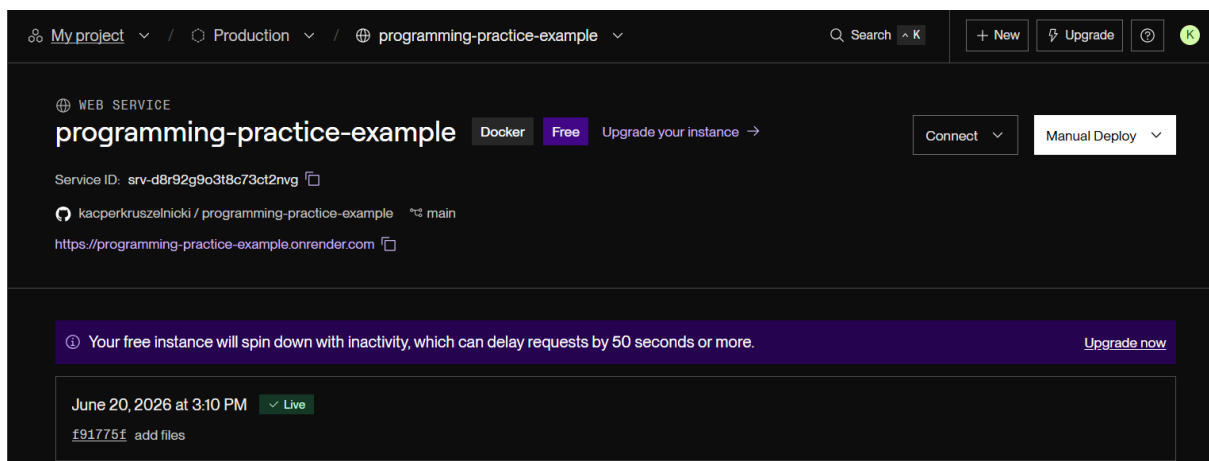


Rysunek 8: Kontener został rzeczywiście uruchomiony i można odpalić aplikację.

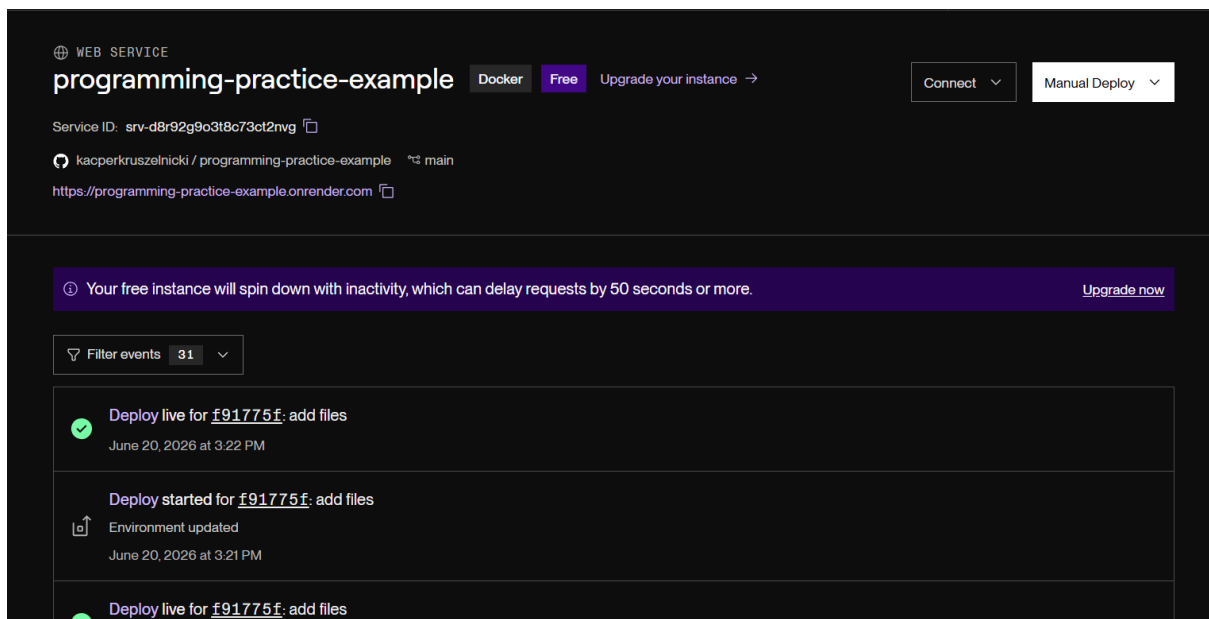
3 Docker Hub



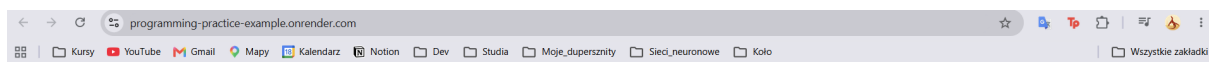
Rysunek 9: Wypchnięty obraz na chmurę.



Rysunek 10: Wdrożenie obrazu na chmurę Renderer.



Rysunek 11: Dodanie instancji bazy danych.



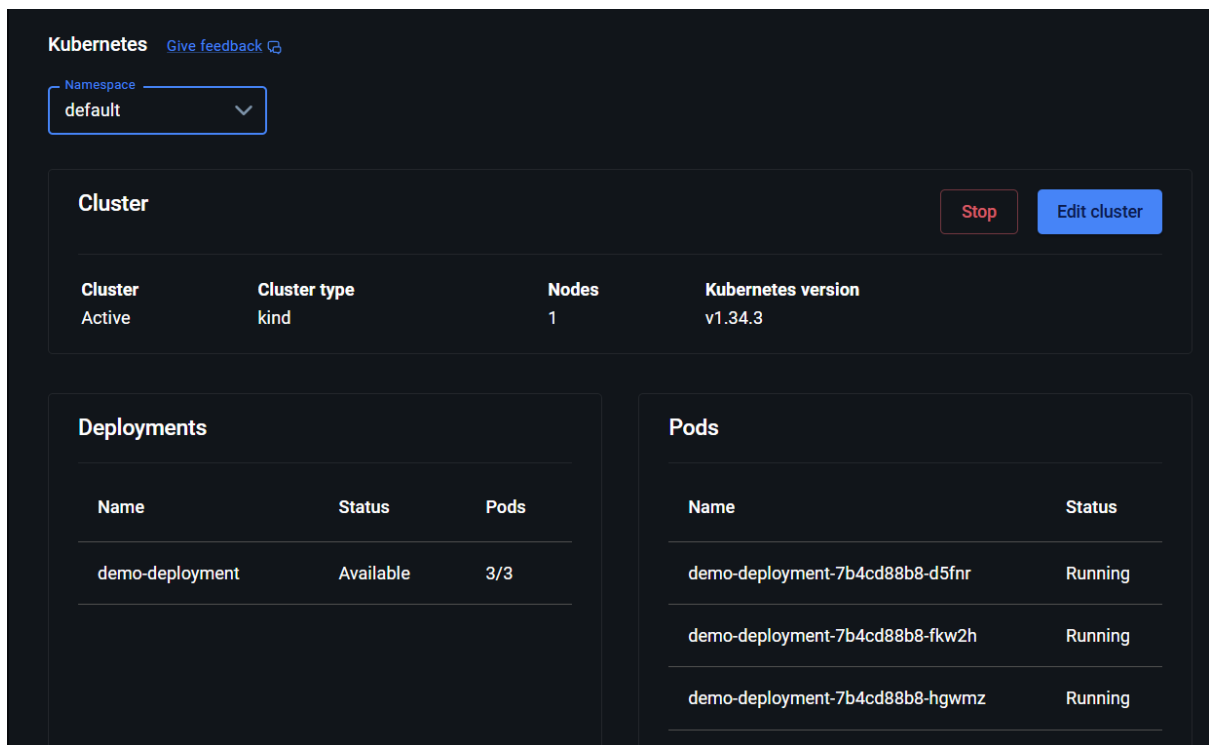
Cześć! Ta strona była odwiedzona 8 razy.

Rysunek 12: Odpalona aplikacja z chmury.

4 Kubernetes

```
PS C:\Users\krusz\GitHub\Programming-Practice\06-Docker> kubectl apply -f k8s-deployment.yaml
deployment.apps/demo-deployment created
service/demo-service created
PS C:\Users\krusz\GitHub\Programming-Practice\06-Docker> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
demo-deployment-7b4cd88b8-d5fnr     1/1     Running   0           15s
demo-deployment-7b4cd88b8-fkw2h     1/1     Running   0           15s
demo-deployment-7b4cd88b8-hgwmz     1/1     Running   0           15s
PS C:\Users\krusz\GitHub\Programming-Practice\06-Docker> kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
demo-deployment-7b4cd88b8-d5fnr     1/1     Running   0           33s   10.244.0.5    desktop-contr
ol-plane <none>                     <none>
demo-deployment-7b4cd88b8-fkw2h     1/1     Running   0           33s   10.244.0.6    desktop-contr
ol-plane <none>                     <none>
demo-deployment-7b4cd88b8-hgwmz     1/1     Running   0           33s   10.244.0.7    desktop-contr
ol-plane <none>                     <none>
```

Rysunek 13: Uruchomienie klastra i wylistowanie podów.



Rysunek 14: Dowód w Docker Desktop na poprawnie działające pody.

```
PS C:\Users\krusz\GitHub\Programming-Practice\06-Docker> kubectl describe pod demo-deployment-7b4cd88b8-fkw2h
Namespace:      default
Priority:        0
Service Account: default
Node:           desktop-control-plane/172.19.0.2
Start Time:     Sat, 20 Jun 2026 15:50:31 +0200
Labels:         app=demo-app
                pod-template-hash=7b4cd88b8
Annotations:    <none>
Status:         Running
IP:             10.244.0.6
IPs:
  IP:           10.244.0.6
Controlled By:  ReplicaSet/demo-deployment-7b4cd88b8
Containers:
  demo-container:
    Container ID:  containerd://2c0d1330e12c6fda3a2f03f8f51ee92ea512be225487f44206b3537444c43dca
    Image:         containous/whoami
    Image ID:      docker.io/containous/whoami@sha256:7d6a3c8f91470a23ef380320609ee6e69ac68d20bc804f3a1c6065fb56cfa34e
    Port:         80/TCP
    Host Port:    0/TCP
    State:        Running
      Started:    Sat, 20 Jun 2026 15:50:36 +0200
    Ready:        True
    Restart Count: 0
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-t8bq9 (ro)
Conditions:
  Type                     Status
  PodReadyToStartContainers True
  Initialized              True
  Ready                    True
  ContainersReady         True
  PodScheduled             True
Volumes:
  kube-api-access-t8bq9:
    Type:              Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:      kube-root-ca.crt
    Optional:           false
```

Rysunek 15: Więcej informacji na temat konkretnego poda.

Kubernetes
[Give feedback](#)

Namespace
default

Cluster

Stop

Edit cluster

Cluster	Cluster type	Nodes	Kubernetes version
Active	kind	1	v1.34.3

Deployments

Name	Status	Pods
demo-deployment	Available	5/5

Pods

Name	Status
demo-deployment-7b4cd88b8-fkw2h	Running
demo-deployment-7b4cd88b8-hgwmz	Running
demo-deployment-7b4cd88b8-gkgsi	Running

Rysunek 16: Zwiększenie liczby dostępnych podów oraz dowód na uzupełnienie usuniętego wcześniej poda.

5 Wnioski

Laboratorium udowadnia, że konteneryzacja za pomocą Dockera eliminuje problemy z niezgodnością środowisk, pozwalając na łatwe i powtarzalne uruchamianie aplikacji. Z kolei wykorzystanie chmury oraz orkiestracji Kubernetes zapewnia niezawodność usług, między innymi dzięki automatycznemu równoważeniu obciążenia i funkcji samonaprawy klastra po awarii. Zastosowanie tych wszystkich narzędzi idealnie obrazuje praktyczne podejście DevOps, które stawia na automatyzację procesów i bezproblemowe, ciągle wdrażanie oprogramowania.